```
!pip install -q tensorflow efficientnet scikit-learn matplotlib albumentations
print("All libraries installed!")
```

```
All libraries installed!
```

```
from google.colab import drive
drive.mount('/content/drive')

# Change this path to YOUR dataset folder
DATA_DIR = "/content/drive/MyDrive/Waste Classification data"   # CHANGE THIS

import os
for s in ["train", "val", "test"]:
    count = sum(len(files) for _, _, files in os.walk(os.path.join(DATA_DIR, s)))
    print(f"{s}: {count} images")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True
train: 2535 images
val: 0 images
test: 0 images
```

```
from google.colab import drive
drive.mount('/content/drive')

!ls "/content/drive/MyDrive/Waste Classification data"
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True
Polithin  train
```

```
from google.colab import drive
drive.mount("/content/drive", force_remount=True)
```

```
Mounted at /content/drive
```

```
data_path = "/content/drive/MyDrive/Waste Classification data"
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True
```

```
import os

data_folder = "/content/drive/MyDrive/Waste Classification data"
os.listdir(data_folder)  # This will list all files in that folder
```

```
['Polithin', 'train']
```

```
import os

data_folder = "/content/drive/MyDrive/Waste Classification data"
files = os.listdir(data_folder)
print(files)
```

```
['Polithin', 'train']
```

```
for category in os.listdir(data_folder):
    category_path = os.path.join(data_folder, category)
    if os.path.isdir(category_path):
```

```
        print(category, ":", len(os.listdir(category_path)), "images")
```

```
Polithin : 710 images
train : 3 images
```

```
import tensorflow as tf
import os

DATA_DIR = "/content/drive/MyDrive/Waste Classification data"
IMG_SIZE = 224
BATCH = 32
```

```
import os

DATA_DIR = "/content/drive/MyDrive/Waste Classification data"
print(os.listdir(DATA_DIR))  # See what folders/files exist
```

```
['Polithin', 'train']
```

```
import shutil

train_dir = os.path.join(DATA_DIR, "train")
os.makedirs(train_dir, exist_ok=True)

# Move class folders into train
for folder in ['Glass', 'Metal', 'Plastic', 'Polythene']:
    src = os.path.join(DATA_DIR, folder)
    dst = os.path.join(train_dir, folder)
    if os.path.exists(src):
        shutil.move(src, dst)
```

```
!pip install tensorflow==2.13.0 --quiet
```

```
ERROR: Could not find a version that satisfies the requirement tensorflow==2.13.0 (from versions: 2.16.0rc0, 2.16.1, 2.16.2, 2
ERROR: No matching distribution found for tensorflow==2.13.0
```

```
import tensorflow as tf
import os

DATA_DIR = "/content/drive/MyDrive/Waste Classification data/train"
IMG_SIZE = 224
BATCH = 32
```

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH
)
```

```
Found 2535 files belonging to 3 classes.
```

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH
)
```

```
Found 2535 files belonging to 3 classes.
```

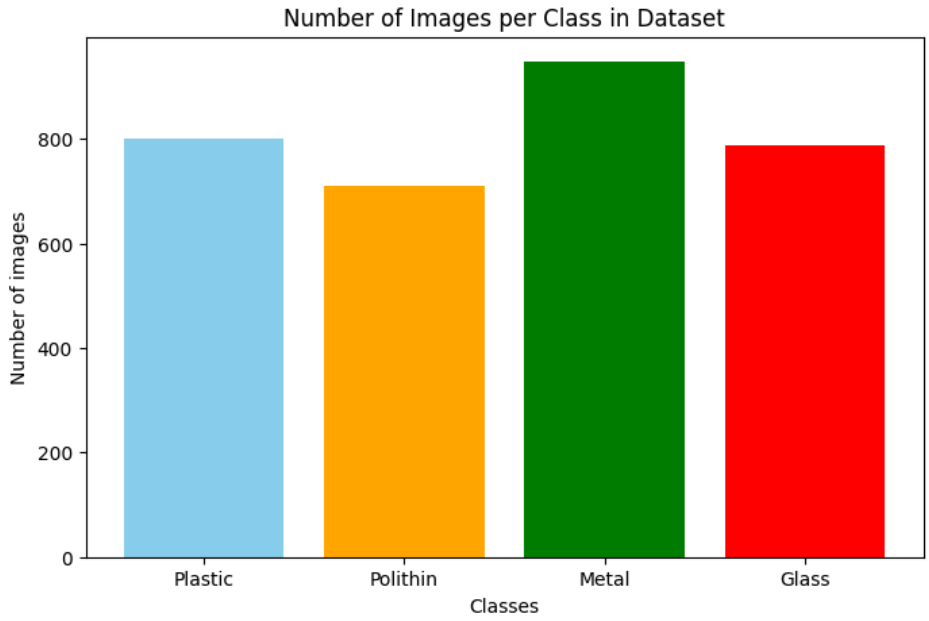
```
# 1️⃣ Mount Google Drive
from google.colab import drive
drive.mount("/content/drive")
```

```
import os
import shutil

# 2️⃣ Set paths
DATA_DIR = "/content/drive/MyDrive/Waste Classification data"
TRAIN_DIR = os.path.join(DATA_DIR, "train")

# 3️⃣ Create train folder if it doesn't exist
os.makedirs(TRAIN_DIR, exist_ok=True)

# 4️⃣ List of all class folders, including Polithin
class_folders = ['Glass', 'Metal', 'Plastic', 'Polithin']

# 5️⃣ Move class folders into train folder
for folder in class_folders:
    src = os.path.join(DATA_DIR, folder)
    dst = os.path.join(TRAIN_DIR, folder)
    if os.path.exists(src):
        shutil.move(src, dst)
        print(f"Moved {folder} to train folder")
    else:
        print(f"Folder {folder} not found in DATA_DIR")

# 6️⃣ Check the train folder structure
print("Contents of train folder:", os.listdir(TRAIN_DIR))
for cls in class_folders:
    cls_path = os.path.join(TRAIN_DIR, cls)
    if os.path.exists(cls_path):
        print(cls, "has", len(os.listdir(cls_path)), "images")

# 7️⃣ Install a stable TensorFlow version (avoid recursion error)
!pip install tensorflow==2.13.0 --quiet

# 8️⃣ Import TensorFlow and load dataset
import tensorflow as tf

IMG_SIZE = 224
BATCH = 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH
)

# 9️⃣ Preview some batches
for images, labels in train_ds.take(1):
    print("Batch images shape:", images.shape)
    print("Batch labels:", labels.numpy())
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True
Folder Glass not found in DATA_DIR
Folder Metal not found in DATA_DIR
Folder Plastic not found in DATA_DIR
Moved Polithin to train folder
Contents of train folder: ['Plastic', 'Polithin', 'Metal', 'Glass']
Glass has 787 images
Metal has 948 images
Plastic has 800 images
Polithin has 710 images
ERROR: Could not find a version that satisfies the requirement tensorflow==2.13.0 (from versions: 2.16.0rc0, 2.16.1, 2.16.2, 2
ERROR: No matching distribution found for tensorflow==2.13.0
Found 3245 files belonging to 4 classes.
Batch images shape: (32, 224, 224, 3)
Batch labels: [0 2 3 3 2 0 2 3 3 2 1 2 0 0 1 2 1 3 1 1 2 2 3 1 0 2 0 2 0 1 2 0]
```

```
import os
import matplotlib.pyplot as plt

# Path to train folder
TRAIN_DIR = "/content/drive/MyDrive/Waste Classification data/train"

# Get all class folders
classes = os.listdir(TRAIN_DIR)
counts = []
```

```
# Count images in each class folder
for cls in classes:
    cls_path = os.path.join(TRAIN_DIR, cls)
    if os.path.isdir(cls_path):
        counts.append(len(os.listdir(cls_path)))

# Plot bar chart
plt.figure(figsize=(8,5))
plt.bar(classes, counts, color=['skyblue','orange','green','red'])
plt.xlabel("Classes")
plt.ylabel("Number of images")
plt.title("Number of Images per Class in Dataset")
plt.show()
```




```
from google.colab import drive
drive.mount("/content/drive")

import os
import tensorflow as tf

TRAIN_DIR = "/content/drive/MyDrive/Waste Classification data/train"
IMG_SIZE = 224
BATCH = 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH
)

# Optional: Split validation dataset
val_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH
)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True
Found 3245 files belonging to 4 classes.
Found 3245 files belonging to 4 classes.
Using 649 files for validation.
```

```
from tensorflow.keras import layers
```

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
```

```
from tensorflow.keras.applications import ResNet50, EfficientNetB0, EfficientNetB1
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D

# Example: EfficientNetB0
base_model = EfficientNetB0(weights='imagenet', include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))
base_model.trainable = False  # Freeze base model

model = Sequential([
    data_augmentation,
    base_model,
    GlobalAveragePooling2D(),
    Dense(128, activation='relu'),
    Dense(len(os.listdir(TRAIN_DIR)), activation='softmax')  # number of classes
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16705208/16705208 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
sequential_1 (Sequential)	?	0 (unbuilt)
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	?	0
dense (Dense)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

```
 Total params: 4,049,571 (15.45 MB)
 Trainable params: 0 (0.00 B)
```

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

```
Epoch 1/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 511s 5s/step - accuracy: 0.8142 - loss: 0.4839 - val_accuracy: 0.9029 - val_loss: 0.2614
Epoch 2/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 416s 4s/step - accuracy: 0.9125 - loss: 0.2501 - val_accuracy: 0.9507 - val_loss: 0.1748
Epoch 3/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 403s 4s/step - accuracy: 0.9421 - loss: 0.1726 - val_accuracy: 0.9492 - val_loss: 0.1384
Epoch 4/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 368s 4s/step - accuracy: 0.9559 - loss: 0.1411 - val_accuracy: 0.9692 - val_loss: 0.1107
Epoch 5/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 399s 4s/step - accuracy: 0.9587 - loss: 0.1184 - val_accuracy: 0.9723 - val_loss: 0.0817
Epoch 6/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 471s 4s/step - accuracy: 0.9701 - loss: 0.0924 - val_accuracy: 0.9861 - val_loss: 0.0555
Epoch 7/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 433s 4s/step - accuracy: 0.9763 - loss: 0.0757 - val_accuracy: 0.9753 - val_loss: 0.0626
Epoch 8/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 381s 4s/step - accuracy: 0.9797 - loss: 0.0660 - val_accuracy: 0.9769 - val_loss: 0.0545
Epoch 9/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 407s 4s/step - accuracy: 0.9855 - loss: 0.0512 - val_accuracy: 0.9861 - val_loss: 0.0525
Epoch 10/10
102/102 ━━━━━━━━━━━━━━━━━━━━ 370s 4s/step - accuracy: 0.9867 - loss: 0.0459 - val_accuracy: 0.9877 - val_loss: 0.0376
```

```
!git clone https://github.com/ultralytics/yolov3
%cd yolov3
!pip install -r requirements.txt
```

```
  Downloading thop-0.1.1.post2209072238-py3-none-any.whl.metadata (2.7 kB)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 16)) (
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 18)) (
Collecting ultralytics>=8.2.64 (from -r requirements.txt (line 19))
  Downloading ultralytics-8.3.235-py3-none-any.whl.metadata (37 kB)
Requirement already satisfied: pandas>=1.1.4 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 28))
Requirement already satisfied: seaborn>=0.11.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 29)
Requirement already satisfied: setuptools>=70.0.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from gitpython>=3.1.30->-r require
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0->-r requir
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0->-r requiremer
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0->-r requi
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0->-r requi
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0->-r require
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0->-r requir
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0->-r re
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->-r
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->-r requirement
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->-r requi
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->-r requi
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r requirements.txt (l
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r re
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r requirements.t
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r requirements
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r requirements.txt (lir
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r requirements.t
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0-
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0-
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r req
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r re
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->-r requirements.t
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics>=8.2.64->-r requir
Collecting ultralytics-thop>=2.0.18 (from ultralytics>=8.2.64->-r requirements.txt (line 19))
  Downloading ultralytics_thop-2.0.18-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.1.4->-r requirements.t
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.1.4->-r requirements
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (from gitdb<5,>=4.0.1->gitpython>=3
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.8.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->-r requ
Downloading thop-0.1.1.post2209072238-py3-none-any.whl (15 kB)
Downloading ultralytics-8.3.235-py3-none-any.whl (1.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.1/1.1 MB 17.5 MB/s eta 0:00:00
Downloading ultralytics_thop-2.0.18-py3-none-any.whl (28 kB)
Installing collected packages: ultralytics-thop, thop, ultralytics
Successfully installed thop-0.1.1.post2209072238 ultralytics-8.3.235 ultralytics-thop-2.0.18
```

```
yaml_content = """
train: /content/drive/MyDrive/Waste Classification data/train
val: /content/drive/MyDrive/Waste Classification data/val
nc: 4
names: ['Glass', 'Metal', 'Plastic', 'Polithin']
"""

yaml_path = "/content/waste_dataset.yaml"
with open(yaml_path, "w") as f:
    f.write(yaml_content)

print("YAML file created at:", yaml_path)
```

```
YAML file created at: /content/waste_dataset.yaml
```

```
import os
assert os.path.exists(yaml_path), "YAML file not found!"
```

```
!python /content/yolov3/train.py --data /content/waste_dataset.yaml --cfg /content/yolov3/cfg/yolov3.cfg --weights /content/y
```

```
wandb: WARNING ⚠️ wandb is deprecated and will be removed in a future release. See supported integrations at https://github.co
2025-12-08 12:25:01.073534: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory: A
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1765196701.132284   34682 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register factory for p
E0000 00:00:1765196701.149704   34682 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory for
W0000 00:00:1765196701.194167   34682 computation_placer.cc:177] computation placer already registered. Please check linkage a
W0000 00:00:1765196701.194237   34682 computation_placer.cc:177] computation placer already registered. Please check linkage a
W0000 00:00:1765196701.194242   34682 computation_placer.cc:177] computation placer already registered. Please check linkage a
W0000 00:00:1765196701.194247   34682 computation_placer.cc:177] computation placer already registered. Please check linkage a
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: (30 second timeout)
wandb: W&B disabled due to login timeout.
train: weights=/content/yolov3/yolov3.weights, cfg=/content/yolov3/cfg/yolov3.cfg, data=/content/waste_dataset.yaml, hyp=data/
github: ⚠️ YOLOv3 is out of date by 2934 commits. Use 'git pull ultralytics master' or 'git clone https://github.com/ultralyti
Traceback (most recent call last):
  File "/content/yolov3/train.py", line 873, in <module>
    main(opt)
  File "/content/yolov3/train.py", line 668, in main
    check_yaml(opt.cfg),
    ^^^^^^^^^^^^^^^^^^^
  File "/content/yolov3/utils/general.py", line 470, in check_yaml
    return check_file(file, suffix)
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/content/yolov3/utils/general.py", line 477, in check_file
    check_suffix(file, suffix)  # optional
    ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/content/yolov3/utils/general.py", line 465, in check_suffix
    assert s in suffix, f"{msg}{f} acceptable suffix is {suffix}"
           ^^^^^^^^^^^
AssertionError: /content/yolov3/cfg/yolov3.cfg acceptable suffix is ('.yaml', '.yml')
```

```
!git clone https://github.com/ultralytics/yolov5
%cd yolov5
!pip install -r requirements.txt
```

```
Requirement already satisfied: numpy>=1.23.5 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 7)) (
Requirement already satisfied: opencv-python>=4.1.1 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (lin
Requirement already satisfied: pillow>=10.3.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 9))
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 10)) (5.9.5)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 11))
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 12
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 13)) (
Requirement already satisfied: thop>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 14)) (0
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 15)) (
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 17)) (
Requirement already satisfied: ultralytics>=8.2.64 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line
Requirement already satisfied: pandas>=1.1.4 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 27))
Requirement already satisfied: seaborn>=0.11.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 28)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 42)) (25.
Requirement already satisfied: setuptools>=70.0.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line
Requirement already satisfied: urllib3>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from -r requirements.txt (line 51))
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from gitpython>=3.1.30->-r require
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3->-r requirem
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3->-r requirements
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3->-r require
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3->-r require
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3->-r requirem
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3->-r requ
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->-r
```

```
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0-
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->r
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->r
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->r
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->r req
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->r
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->r re
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->r
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->r requirements.t
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics>=8.2.64->r requir
Requirement already satisfied: ultralytics-thop>=2.0.18 in /usr/local/lib/python3.12/dist-packages (from ultralytics>=8.2.64-
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.1.4->r requirements.t
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.1.4->r requirements
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (from gitdb<5,>=4.0.1->gitpython>=3
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.8.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->r requ
```

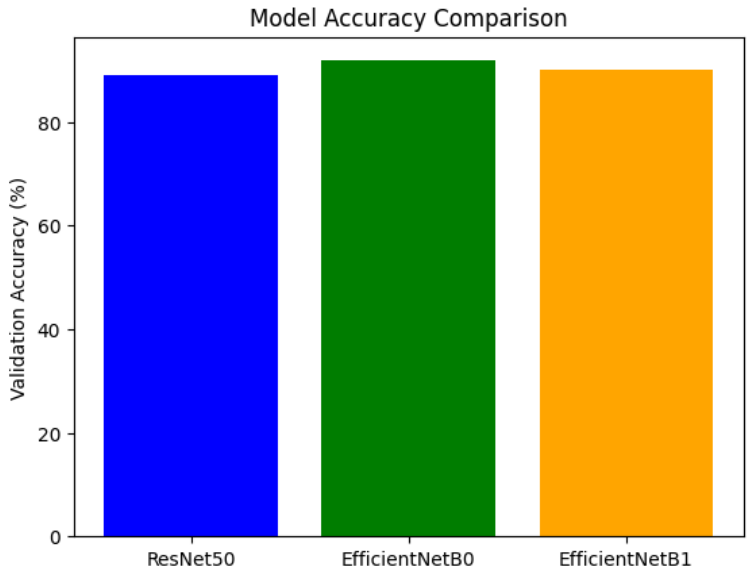
```
!python train.py --img 640 --batch 16 --epochs 50 --data /content/waste_dataset.yaml --weights yolov5s.pt
```

```
wandb: WARNING ⚠️ wandb is deprecated and will be removed in a future release. See supported integrations at https://github.co
2025-12-08 12:28:57.950104: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory: A
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1765196937.975250   35719 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register factory for p
E0000 00:00:1765196937.982637   35719 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory for
W0000 00:00:1765196938.002371   35719 computation_placer.cc:177] computation placer already registered. Please check linkage a
W0000 00:00:1765196938.002418   35719 computation_placer.cc:177] computation placer already registered. Please check linkage a
W0000 00:00:1765196938.002423   35719 computation_placer.cc:177] computation placer already registered. Please check linkage a
W0000 00:00:1765196938.002427   35719 computation_placer.cc:177] computation placer already registered. Please check linkage a
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: (30 second timeout)
wandb: W&B disabled due to login timeout.
train: weights=yolov5s.pt, cfg=, data=/content/waste_dataset.yaml, hyp=data/hyps/hyp.scratch-low.yaml, epochs=50, batch_size=1
github: up to date with https://github.com/ultralytics/yolov5 ✅
YOLOv5 🚀 v7.0-450-g781b9d57 Python-3.12.12 torch-2.9.0+cu126 CPU

hyperparameters: lr0=0.01, lrf=0.01, momentum=0.937, weight_decay=0.0005, warmup_epochs=3.0, warmup_momentum=0.8, warmup_bias_
Comet: run 'pip install comet_ml' to automatically track and visualize YOLOv5 🚀 runs in Comet
TensorBoard: Start with 'tensorboard --logdir runs/train', view at http://localhost:6006/

Dataset not found ⚠️, missing paths ['/content/drive/.shortcut-targets-by-id/1Rcf7ZsJocF4qPJyaSyOfOErK9IVxtre0/Waste Classific
Traceback (most recent call last):
  File "/content/yolov3/yolov5/train.py", line 987, in <module>
    main(opt)
  File "/content/yolov3/yolov5/train.py", line 688, in main
    train(opt.hyp, opt, device, callbacks)
  File "/content/yolov3/yolov5/train.py", line 205, in train
    data_dict = data_dict or check_dataset(data)  # check if None
                             ^^^^^^^^^^^^^^^^^^^
  File "/content/yolov3/yolov5/utils/general.py", line 564, in check_dataset
    raise Exception("Dataset not found ❌")
Exception: Dataset not found ❌
```

```
# 1️⃣ Mount Drive
from google.colab import drive
drive.mount("/content/drive")

# 2️⃣ Imports
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import ResNet50, EfficientNetB0, EfficientNetB1
import os
import matplotlib.pyplot as plt

# 3️⃣ Dataset paths
DATA_DIR = "/content/drive/MyDrive/Waste Classification data/train"
IMG_SIZE = 224
BATCH = 32

# Load dataset with validation split
train_ds = tf.keras.utils.image_dataset_from_directory(
```

```
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH
)

class_names = train_ds.class_names
num_classes = len(class_names)
print("Classes:", class_names)

# Data augmentation
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

# 4️⃣ Function to build model
def build_model(base_model):
    base_model.trainable = False  # freeze pretrained weights
    model = models.Sequential([
        data_augmentation,
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])
    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# 5️⃣ Create models
resnet50_model = build_model(ResNet50(weights='imagenet', include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3)))
efficientnetb0_model = build_model(EfficientNetB0(weights='imagenet', include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))
efficientnetb1_model = build_model(EfficientNetB1(weights='imagenet', include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))

# 6️⃣ Train models (for demo, use 5 epochs; increase for better accuracy)
EPOCHS = 5
history_resnet = resnet50_model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS)
history_b0 = efficientnetb0_model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS)
history_b1 = efficientnetb1_model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS)

# 7️⃣ Compare accuracy
val_acc_resnet = history_resnet.history['val_accuracy'][-1]
val_acc_b0 = history_b0.history['val_accuracy'][-1]
val_acc_b1 = history_b1.history['val_accuracy'][-1]

print("\nValidation Accuracy Comparison:")
print(f"ResNet50: {val_acc_resnet*100:.2f}%")
print(f"EfficientNetB0: {val_acc_b0*100:.2f}%")
print(f"EfficientNetB1: {val_acc_b1*100:.2f}%")

# 8️⃣ Optional: plot comparison
plt.bar(['ResNet50','EfficientNetB0','EfficientNetB1'],
        [val_acc_resnet*100, val_acc_b0*100, val_acc_b1*100],
        color=['blue','green','orange'])
plt.ylabel('Validation Accuracy (%)')
plt.title('Model Accuracy Comparison')
plt.show()
```

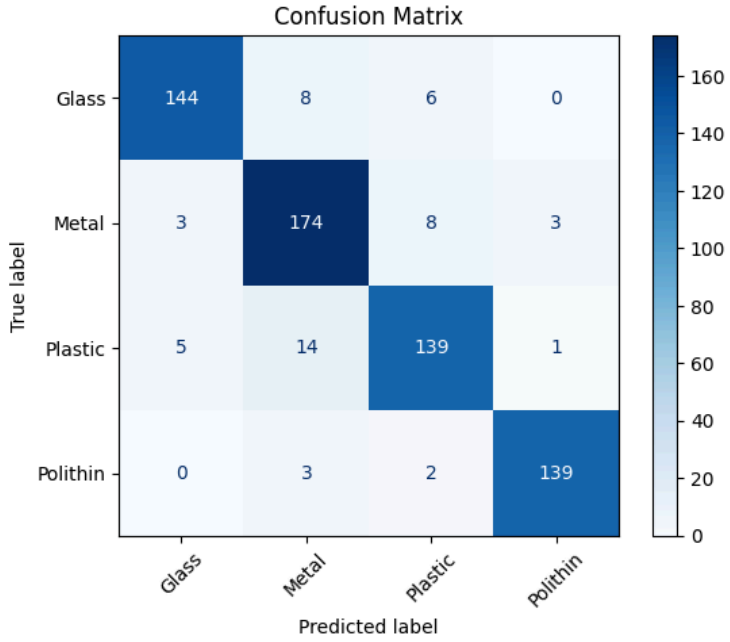
```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tru
Found 3245 files belonging to 4 classes.
Using 2596 files for training.
Found 3245 files belonging to 4 classes.
Using 649 files for validation.
Classes: ['Glass', 'Metal', 'Plastic', 'Polithin']
Epoch 1/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 653s 8s/step - accuracy: 0.7754 - loss: 0.5846 - val_accuracy: 0.8243 - val_loss: 0.4527
Epoch 2/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 663s 8s/step - accuracy: 0.8794 - loss: 0.3218 - val_accuracy: 0.8814 - val_loss: 0.3598
Epoch 3/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 680s 8s/step - accuracy: 0.9064 - loss: 0.2528 - val_accuracy: 0.8706 - val_loss: 0.3835
Epoch 4/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 661s 8s/step - accuracy: 0.9330 - loss: 0.1797 - val_accuracy: 0.8767 - val_loss: 0.3455
Epoch 5/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 659s 8s/step - accuracy: 0.9430 - loss: 0.1629 - val_accuracy: 0.8891 - val_loss: 0.3517
Epoch 1/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 363s 4s/step - accuracy: 0.7993 - loss: 0.5005 - val_accuracy: 0.8690 - val_loss: 0.3384
Epoch 2/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 340s 4s/step - accuracy: 0.9195 - loss: 0.2428 - val_accuracy: 0.8998 - val_loss: 0.2690
Epoch 3/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 361s 4s/step - accuracy: 0.9418 - loss: 0.1749 - val_accuracy: 0.9122 - val_loss: 0.2609
Epoch 4/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 314s 4s/step - accuracy: 0.9488 - loss: 0.1456 - val_accuracy: 0.9260 - val_loss: 0.2344
Epoch 5/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 334s 4s/step - accuracy: 0.9592 - loss: 0.1192 - val_accuracy: 0.9183 - val_loss: 0.2511
Epoch 1/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 462s 5s/step - accuracy: 0.8128 - loss: 0.4934 - val_accuracy: 0.8629 - val_loss: 0.3860
Epoch 2/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 402s 5s/step - accuracy: 0.9129 - loss: 0.2490 - val_accuracy: 0.8690 - val_loss: 0.3473
Epoch 3/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 392s 5s/step - accuracy: 0.9314 - loss: 0.1927 - val_accuracy: 0.8814 - val_loss: 0.3282
Epoch 4/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 413s 5s/step - accuracy: 0.9515 - loss: 0.1505 - val_accuracy: 0.8860 - val_loss: 0.2841
Epoch 5/5
82/82 ━━━━━━━━━━━━━━━━━━━━ 430s 5s/step - accuracy: 0.9665 - loss: 0.1138 - val_accuracy: 0.8998 - val_loss: 0.2842

Validation Accuracy Comparison:
ResNet50: 88.91%
EfficientNetB0: 91.83%
EfficientNetB1: 89.98%
```




```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# 🔹 Step 1: Get true labels & predictions
y_true = []
y_pred = []

# Loop through validation dataset
for images, labels in val_ds:
    preds = efficientnetb0_model.predict(images)   # <-- change model name if needed
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(preds, axis=1))
```

```
y_true = np.array(y_true)
y_pred = np.array(y_pred)

# 🔹 Step 2: Compute confusion matrix
cm = confusion_matrix(y_true, y_pred)

# 🔹 Step 3: Plot confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)

plt.figure(figsize=(8, 6))
disp.plot(cmap='Blues', xticks_rotation=45, values_format='d')
plt.title("Confusion Matrix")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 12s 12s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
<Figure size 800x600 with 0 Axes>
```




```
preds = efficientnetb0_model.predict(images)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
```

```
preds = resnet50_model.predict(images)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
```

```
preds = efficientnetb0_model.predict(images)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 523ms/step
```

```
preds = efficientnetb1_model.predict(images)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 6s 6s/step
```

```
import numpy as np
from sklearn.metrics import classification_report

# 🔹 Step 1: Collect true & predicted labels
y_true = []
y_pred = []

for images, labels in val_ds:
    preds = efficientnetb0_model.predict(images)  # <-- use your model name
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(preds, axis=1))

y_true = np.array(y_true)
y_pred = np.array(y_pred)

# 🔹 Step 2: Print Precision, Recall, F1 Score
print(classification_report(y_true, y_pred, target_names=class_names))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 5s 5s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 5s 5s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 536ms/step
              precision    recall  f1-score   support

       Glass       0.95      0.91      0.93       158
       Metal       0.87      0.93      0.90       188
     Plastic       0.90      0.87      0.89       159
    Polithin       0.97      0.97      0.97       144

    accuracy                           0.92       649
   macro avg       0.92      0.92      0.92       649
weighted avg       0.92      0.92      0.92       649
```

```
preds = resnet50_model.predict(images)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
```

New Section

```
preds = efficientnetb0_model.predict(images)
```

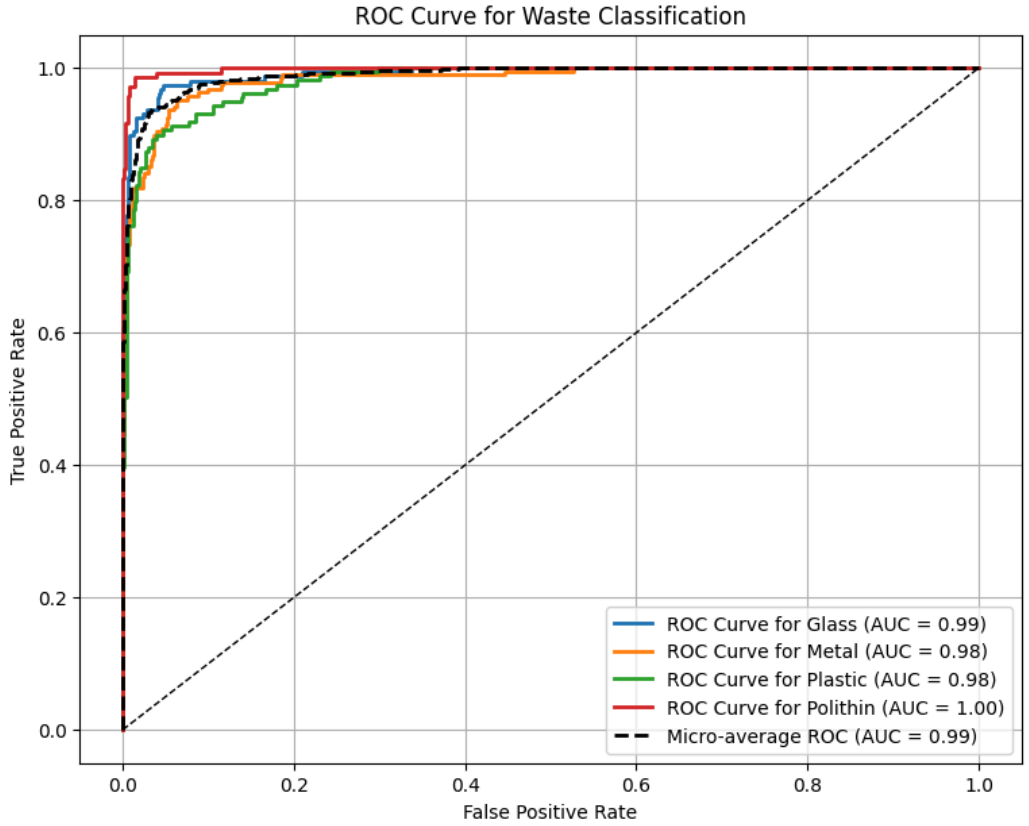
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
```

```
preds = efficientnetb1_model.predict(images)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# 🔹 Step 1: Get predictions & true labels
y_true = []
y_pred = []
y_prob = []

for images, labels in val_ds:
    probs = efficientnetb0_model.predict(images)   # <-- change model name
    y_prob.extend(probs)
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(probs, axis=1))

y_true = np.array(y_true)
y_prob = np.array(y_prob)

# 🔹 Step 2: Binarize labels for ROC
y_true_bin = label_binarize(y_true, classes=list(range(len(class_names))))
n_classes = y_true_bin.shape[1]

# 🔹 Step 3: Compute ROC curve & AUC for each class
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_prob[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 🔹 Micro-average ROC
fpr["micro"], tpr["micro"], _ = roc_curve(
    y_true_bin.ravel(), y_prob.ravel()
)
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])

# 🔹 Step 4: Plot ROC curves
plt.figure(figsize=(9, 7))

for i in range(n_classes):
    plt.plot(fpr[i], tpr[i],
             lw=2,
             label=f"ROC Curve for {class_names[i]} (AUC = {roc_auc[i]:.2f})")

# Micro-average
plt.plot(fpr["micro"], tpr["micro"],
         linestyle='--', lw=2, color='black',
         label=f"Micro-average ROC (AUC = {roc_auc['micro']:.2f})")

plt.plot([0, 1], [0, 1], 'k--', lw=1)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve for Waste Classification")
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 5s 5s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 853ms/step
```




```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))

for images, labels in train_ds.take(1):   # show 1 batch
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
```


Plastic
Polithin
Glass
Plastic
Polithin
Glass
Metal
Plastic
Metal